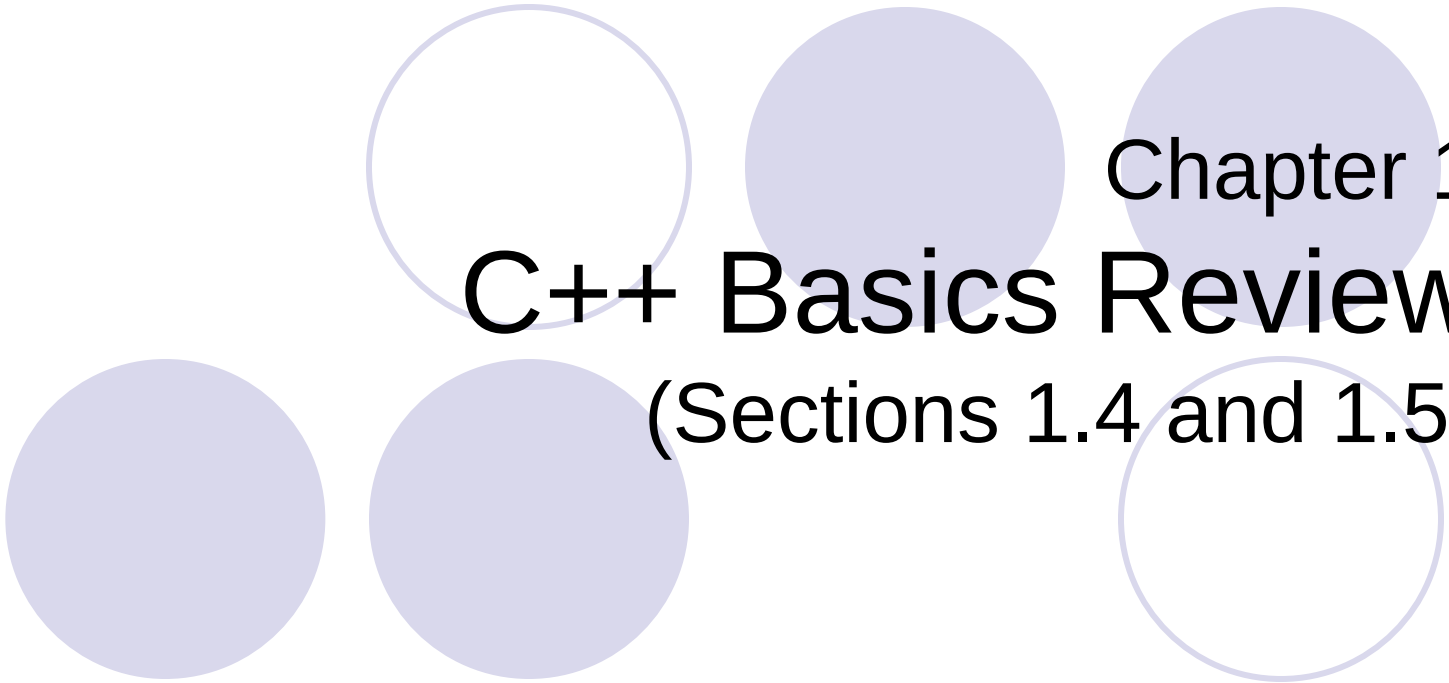


Chapter 1

C++ Basics Review

(Sections 1.4 and 1.5)

Classes

- Defines the organization of a data user-defined type.
- Members can be
  - Data
  - Functions/Methods
- Information Hiding Labels
  - public
  - private
- Constructors
  - We have two in this example
  - Why?

```
1   /**
2    * A class for simulating an integer memory cell.
3    */
4   class IntCell
5   {
6     public:
7       /**
8        * Construct the IntCell.
9        * Initial value is 0.
10       */
11      IntCell( )
12        { storedValue = 0; }
13
14      /**
15       * Construct the IntCell.
16       * Initial value is initialValue.
17       */
18      IntCell( int initialValue )
19        { storedValue = initialValue; }
20
21      /**
22       * Return the stored value.
23       */
24      int read( )
25        { return storedValue; }
26
27      /**
28       * Change the stored value to x.
29       */
30      void write( int x )
31        { storedValue = x; }
32
33    private:
34      int storedValue;
35  };
```

Additional Syntax and Accessors

- Initialization list
  - Init data members directly in the constructor
- Explicit constructor
  - Avoids implicit type conversion

    obj = 37; // Error: type mismatch
- Constant member functions
  - Examines, but does not change the object state
  - Called ‘accessor /getters’
- Non-const functions are called ‘mutators / setters’

```
1   /**
2    * A class for simulating an integer memory cell.
3    */
4   class IntCell
5   {
6     public:
7       explicit IntCell( int initialValue = 0 )
8         : storedValue{ initialValue } { }
9       int read( ) const
10        { return storedValue; }
11      void write( int x )
12        { storedValue = x; }
13
14    private:
15      int storedValue;
16  };
```

Interface and Implementation

- Interface typically defined in .h files
  - `#include` in `.c` file
- Preprocessor commands
  - Guards against multiple inclusion of `.h` files

Interface

```
1  #ifndef IntCell_H
2  #define IntCell_H
3
4  /**
5   * A class for simulating an integer memory cell.
6   */
7  class IntCell
8  {
9    public:
10     explicit IntCell( int initialValue = 0 );
11     int read( ) const;
12     void write( int x );
13
14   private:
15     int storedValue;
16 };
17
18 #endif
```

Interface and Implementation

- Scoping resolution operator
  - ClassName::member
  - To identify the class corresponding to each function
- Remember:
  - Function signatures must match in both interface and implementation
  - Default parameters are specified only in the interface

Implementation

```
1  #include "IntCell.h"
2
3  /**
4   * Construct the IntCell with initialValue
5   */
6  IntCell::IntCell( int initialValue ) : storedValue{ initialValue }
7  {
8  }
9
10 /**
11  * Return the stored value.
12  */
13 int IntCell::read( ) const
14 {
15     return storedValue;
16 }
17
18 /**
19  * Store x.
20  */
21 void IntCell::write( int x )
22 {
23     storedValue = x;
24 }
```

main() function

- Objects are declared just like primitive data types.

- Legal Declarations
  - `Intcell obj1; // zero parameter constructor`
  - `Intcell obj2(12); // one parameter constructor`
  - `Intcell obj1{ }; // C++11`
  - `Intcell obj2{12}; // C++11`
- Illegal declarations
  - `Intcell obj3 = 37;`
  - `Intcell obj4();`

main() function

```
1   #include <iostream>
2   #include "IntCell.h"
3   using namespace std;
4
5   int main( )
6   {
7       IntCell m;
8
9       m.write( 5 );
10      cout << "Cell contents: " << m.read( ) << endl;
11
12      return 0;
13  }
```

Vectors

- Replaces built-in C++ arrays
  - Built-in arrays do not act as proper C++ objects
- `Vector` classes in standard template library STL
  - Gives a `size()` function
  - Can be assigned using `=`
  - Can be compared using `==<>`
  - Initial size 0 and grows as needed

- Similarly C++ also provides standard `string` class.

Vectors

- built-in C++ array

```
int daysInMonth[ ] = { 31, 28, 31, 30, 31, 30, 31, 31, 30,
   31, 30, 31 };
```

- vector in C++11

```
vector<int> daysInMonth( 12 ); // No {} before C++11

vector<int> daysInMonth { 31, 28, 31, 30, 31, 30, 31, 31,
   30, 31, 30, 31};
```

Vectors

```
#include <iostream>
#include <vector>
using namespace std;
int main( )
{
vector<int> squares( 100 );
for(int i=0;i<squares.size();++i)//in C++11: for(auto x:squares)
 squares[ i ] = i * i;

for( int i = 0; i < squares.size( ); ++i )
  cout << i << " " << squares[ i ] << endl;
return 0;}
```

Pointers

- Pointer variable
  - Stores the address of another object in memory.
- Declaration
  - * before the variable name indicates a pointer declaration
  - Pointers are uninitialized at declaration time.
  - Reading uninitialized pointer values results in bugs.
- Dynamic object creation
  - Using the `new` keyword

    m = `new` IntCell( );

    m = `new` IntCell{ }; // C++11

```
1   int main( )
2   {
3       IntCell *m;
4
5       m = new IntCell( 0 );
6       m->write( 5 );
7       cout << "Cell contents: " << m->read( ) << endl;
8
9       delete m;
10      return 0;
11  }
```

Pointers (contd)

- Garbage collection
  - Objects allocated using `new` must be explicitly `delete`d.
  - Else your program will have memory leaks
  - There's no automatic GC in C++.
- Accessing members of an object
  - Use the `->` operator
- Address-of operator
  - `&obj` gives the address where `obj` is stored.

```
1   int main( )
2   {
3       IntCell *m;
4
5       m = new IntCell( 0 );
6       m->write( 5 );
7       cout << "Cell contents: " << m->read( ) << endl;
8
9       delete m;
10      return 0;
11  }
```

Lvalues, Rvalues, and References

- **lvalue (&)**

identifies a non-temporary object.

- **rvalue (&&)**

identifies a temporary object or is
a value (such as a literal constant)
lvalues: arr, str, arr[x], x, y, z, ptr, *ptr
rvalues: 2, "foo", x+y

```
vector<string> arr( 3 );
const int x = 2;
int y;
  ...
int z = x + y;
string str = "foo";
vector<string> *ptr = &arr;
```

- **References**
  - **lvalue references**
  - **rvalue references // C++11**
- Reference variables are often used to avoid copying objects across function-call boundaries

```
string str = "hell";
string & rstr = str;
rstr += 'o';
```

```
string str = "hell";
string && bad1 = "hello";
```

Parameter Passing

- **1. Call-by-value** is appropriate for small objects that should not be altered by the function.
- **2. Call-by-reference** is appropriate for all objects that may be altered by the function.
- **3. Call-by-constant-reference** is appropriate for large objects that should not be altered by the function and are expensive to copy
- **4. call-by-rvalue-reference (C++11)**

```
double average( double a, double b );    // returns average of a and b
void swap( double a, double b );         // swaps a and b; wrong parameter types
string randomItem( vector<string> arr ); // returns a random item in arr; inefficient
```

```
void swap( double & a, double & b );  string randomItem( const vector<string> & arr );
```

```
string randomItem( const vector<string> & arr );  // returns random item in lvalue arr
string randomItem( vector<string> && arr );       // returns random item in rvalue arr

vector<string> v { "hello", "world" };
cout << randomItem( v ) << endl;                      // invokes lvalue method
cout << randomItem( { "hello", "world" } ) << endl; // invokes rvalue method
```

Return Passing

- **Return by value**
  - Makes a copy of the variable returned
- **Return by reference**
  - Return the address of the variable returned
- **Return by constant reference**
  - Return the address of the variable returned
  - Return value cannot be modified by caller.
  - Avoid an immediate copy. Must use a constant reference to access the return value

```
1   LargeType randomItem1( const vector<LargeType> & arr )
2   {
3       return arr[ randomInt( 0, arr.size( ) - 1 ) ];
4   }
5
6   const LargeType & randomItem2( const vector<LargeType> & arr )
7   {
8       return arr[ randomInt( 0, arr.size( ) - 1 ) ];
9   }
10
11      vector<LargeType> vec;
12        ...
13      LargeType item1 = randomItem1( vec );           // copy
14      LargeType item2 = randomItem2( vec );           // copy
15      const LargeType & item3 = randomItem2( vec );   // no copy
```

std::swap and std::move

- Swapping by three copies. The copies are very expensive!

```
void swap( vector<string> & x, vector<string> & y )
{
    vector<string> tmp = x;
    x = y;
    y = tmp;
}
```

- Swapping by three moves. Use rvalue to avoid copies. In C++11

```
void swap( vector<string> & x, vector<string> & y )
{
    vector<string> tmp = std::move( x );
    x = std::move( y );
    y = std::move( tmp );
}
```

The Rule of Big Five:

- In C++11, classes come with five special functions by default: Destructor, Copy Constructor, Move Constructor , Copy Assignment operator=, Move Assignment operator=

```
~IntCell( );                                          // Destructor
IntCell( const IntCell & rhs );                       // Copy constructor
IntCell( IntCell && rhs );                            // Move constructor
IntCell & operator= ( const IntCell & rhs );          // Copy assignment
IntCell & operator= ( IntCell && rhs );               // Move assignment
```

Destructor

- Called whenever
  - Object goes out of scope
  - `delete` called
- Frees up resource allocated for the object

```
1   IntCell::~IntCell( )
2   {
3       // Does nothing, since IntCell contains only an int data
4       // member. If IntCell contained any class objects, their
5       // destructors would be called.
6   }
```

Copy and move constructors

- Initializes a new object to another of its own type
- Invoked during
  - Declaration

```
IntCell B = C; // Copy construct if C is
  lvalue; Move construct if C is rvalue
Intcell B {C}; // Copy construct if C is
  lvalue; Move construct if C  is rvalue
```

  - Call by value
  - Return by value
- But not in `B = C;` (assignment operator)

```
8   IntCell::IntCell( const IntCell & rhs ) : storedValue( rhs.storedValue )
9   {
10  }
```

operator=

- Copy assignment and move assignment

  lhs=rhs

```
11
12  const IntCell & IntCell::operator=( const IntCell & rhs )
13  {
14      if( this != &rhs )   // Standard alias test
15          storedValue = rhs.storedValue;
16      return *this;
17  }
```

Problem with defaults

- Class defaults works well with data members that are primitive types and objects.
- Usually don't work when data member is a pointer type
- In this example, default `operator=` and copy constructor copy the pointer instead of the value. **Shallow copy**
- **Deep copy assignment**
- **Aliasing**
- Explicitly disallow each (**delete**).

```
IntCell( const IntCell & rhs ) = delete;
```

```
1   class IntCell
2   {
3     public:
4       explicit IntCell( int initialValue = 0 )
5         { storedValue = new int( initialValue ); }
6
7       int read( ) const
8         { return *storedValue; }
9       void write( int x )
10        { *storedValue = x; }
11    private:
12      int *storedValue;
13  };
```

```
1  int f( )
2  {
3      IntCell a( 2 );
4      IntCell b = a;
5      IntCell c;
6
7      c = b;
8      a.write( 4 );
9      cout << a.read( ) << endl << b.read( ) << endl << c.read( ) << endl;
10     return 0;
11 }
```